

FenixCMS SaaS

Plan de mejoras adicionales - fases 24 a 44

Repositorio: `fenixcmssl-spec/moreno`

Rama auditada: `main`

Commit de referencia auditado: `3756204d6feab412042c610c8a57e9343c886d0f`

Fecha de auditoría: 30 de septiembre de 2026

Documento: complemento técnico del Plan Maestro FenixCMS FASE 0-23

1. Finalidad de este documento

Este documento añade las mejoras detectadas en una nueva revisión del estado real del repositorio GitHub.

No sustituye el Plan Maestro anterior. Las FASES 0-23 continúan siendo válidas como línea base. Este documento incorpora las FASES 24-44, centradas en cerrar problemas que siguen visibles en `main`, endurecer producción, eliminar caminos de prueba que todavía pueden llegar a negocio y preparar una operación SaaS real.

La regla operativa continúa siendo:

UNA FASE -> UNA IMPLEMENTACIÓN -> UNA VALIDACIÓN -> UN COMMIT -> SOLO DESPUÉS LA SIGUIENTE FASE.

No se debe implementar varias fases de golpe en AI Studio. El objetivo es poder comprobar exactamente qué se ha cambiado y poder volver atrás sin perder trazabilidad.

2. Fotografía técnica actual de GitHub

En el commit auditado se observan aproximadamente:

- 296 rutas/archivos en el árbol del repositorio.
- 63 rutas API con `route.ts`.
- 28 servicios de negocio en `lib/services`.
- 35 modelos Prisma.
- 24 modelos con `tenantId` según la estructura actual.
- 11 archivos de tests.
- 6 migraciones Prisma.
- 1 workflow de GitHub Actions dedicado actualmente a generar el PDF del plan anterior.

El proyecto ya utiliza PostgreSQL + Prisma, sesiones persistentes, aislamiento multi-tenant, planes, entitlements, licencias, pagos, suscripciones, facturas, dominios, temas, plugins, media, productos, pedidos, blog y clasificados.

Sin embargo, la revisión actual detecta una diferencia importante entre la arquitectura declarada y varios caminos concretos del código.

3. Hallazgos adicionales que justifican las nuevas fases

3.1. Resolución de tenant todavía demasiado permisiva

`middleware.ts` acepta `x-simulated-host` y también puede convertir parámetros `tenant`, `store` y `slug` de la URL en contexto de tenant.

`TenantContextHelper.resolvePublicTenant()` también considera parámetros y headers antes de terminar la resolución por dominio, y contiene fallback al primer tenant activo.

Esto no debe utilizarse como autoridad comercial en producción.

3.2. La extensión Prisma no bloquea siempre la ausencia de contexto

En `lib/prisma.ts`, para determinados modelos tenant-scoped, si no existe un `activeTenantId` y tampoco existe un bypass, la implementación actual continúa con `query(args)`.

Eso significa que la garantía de aislamiento depende todavía de que la capa superior haya establecido correctamente el contexto.

Debe existir un modo fail-closed: sin tenant validado, una operación tenant-scoped debe ser rechazada salvo que exista un bypass de sistema explícito.

3.3. El flujo SaaS todavía contiene simulaciones de pago

`lib/services/saas-checkout.service.ts` todavía:

- importa `INITIAL\_PLANS`, `INITIAL\_APPLICATIONS` y `INITIAL\_TENANTS`;
- puede inventar un plan si no existe;
- crea un tenant previo al pago mediante `ensureTenantExists`;
- fabrica URLs de checkout de Stripe/PayPal;

- permite completar el proceso usando una verificación que en la práctica no autentica la transacción real.

Esto debe cerrarse.

3.4. PayPal webhook no está validado criptográficamente de extremo a extremo

``WebhookService.verifyPayPalSignature()`` comprueba headers, timestamp, dominio del certificado y formato Base64, pero no realiza todavía la verificación criptográfica completa de PayPal.

La documentación oficial de PayPal exige verificar la firma, mediante la verificación criptográfica del mensaje o el endpoint oficial de verificación. El mensaje usa el ``transmission_id``, ``transmission_time``, ``webhook_id`` y el hash CRC32 del cuerpo original.

3.5. APPROVED no debe significar pago capturado

El código actual procesa ``CHECKOUT.ORDER.APPROVED`` como evento suficiente para operaciones de licencia.

Para Orders API, la aprobación del comprador y la captura del pago son estados distintos. El aprovisionamiento comercial debe depender de la captura confirmada, normalmente ``PAYMENT.CAPTURE.COMPLETED``.

3.6. Media todavía puede devolver éxito sin persistencia

``app/api/media/route.ts`` tiene rutas de fallback que pueden devolver archivos seed o un objeto de upload aunque el storage/DB no haya persistido realmente el archivo.

``ManagedStorageProvider`` calcula metadata y una URL pero no escribe realmente el contenido físico.

3.7. Rate limiting es local al proceso

``SecurityService`` utiliza un ``Map`` en memoria.

En una sola instancia puede funcionar como protección básica, pero no representa un límite global para varias instancias/procesos y pierde estado al reiniciar.

3.8. El sanitizer global no debe ser la política del CMS

``sanitizePayload()`` elimina etiquetas y patrones de texto de todos los strings.

Para campos de texto enriquecido esto puede destruir contenido válido. La seguridad debe ser contextual: validación de entrada, sanitización HTML con allowlist cuando el campo lo permita y encoding al renderizar.

3.9. Precios monetarios usan Float

El esquema Prisma utiliza ``Float`` para varios importes comerciales.

Para dinero es preferible usar un tipo decimal/numeric o una representación de unidades menores enteras y definir una única política de redondeo.

3.10. Checkout público de pedidos acepta demasiado contexto desde el cliente

``app/api/orders/route.ts`` puede resolver un tenant a partir de información enviada por el cliente si no tiene ya un contexto de dominio/sesión.

El precio, stock, descuento, impuestos y tienda deben ser determinados por servidor.

3.11. Super Admin sigue teniendo datos de memoria y settings no persistentes

Hay getters que consultan PostgreSQL pero, según el método, pueden caer a datos ``INITIAL_`` si no hay resultados o ante excepciones. Los settings de plataforma se mantienen en memoria.

Un panel administrativo de producción debe distinguir:

DB vacía != error de DB != datos demo.

3.12. Entitlements API incompleta

``api/admin/entitlements`` expone GET, pero no existe una superficie equivalente completa de escritura para administrar `PlanEntitlement` como recurso de forma coherente.

3.13. Toolchain inconsistente

El repositorio contiene ``package-lock.json`` y ``bun.lock``, pero ``package.json`` no declara ``packageManager``. Además, el script de tests utiliza ``bunx``.

Esto reduce la reproducibilidad entre el ordenador Debian y otros entornos.

3.14. CI/CD insuficiente

En GitHub sólo se observa el workflow del PDF.

Faltan como política del repositorio, entre otras cosas, un pipeline de build/lint/test, validación de Prisma, auditoría de dependencias y controles de merge.

4. Orden global de las nuevas fases

La prioridad técnica debe seguir este orden:

24 -> 25 -> 26 -> 27 -> 28 -> 29 -> 30 -> 31 -> 32 -> 33 -> 34 -> 35 -> 36 -> 37 -> 38 -> 39 -> 40 -> 41 -> 42 -> 43 -> 44

No saltar directamente al VPS. Primero código y pruebas; después integración de staging; finalmente producción.

FASE 24 - Resolver el tenant exclusivamente por contexto confiable

Ejecución: AI STUDIO + ORDENADOR DEBIAN para revisión

Estado: CRÍTICO

Objetivo

Eliminar la posibilidad de que el navegador pueda escoger arbitrariamente el tenant que representa una petición pública.

Hallazgo

`middleware.ts` admite:

- `x-simulated-host`;
- `?tenant=`;
- `?store=`;
- `?slug=`.

`TenantContextHelper.resolvePublicTenant()` también considera estos valores y tiene fallback al primer tenant activo.

Modificaciones paso a paso

1. Definir una única función central de resolución:

- hostname recibido;
- dominio/subdominio registrado;
- dominio verificado y activo;
- tenant correspondiente.

2. Para producción, eliminar la autoridad de:

- `x-simulated-host`;
- `tenant`;
- `store`;
- `slug`;
- `tenantId` enviados por el navegador en peticiones públicas.

3. Mantener simulación sólo en desarrollo mediante una bandera explícita, por ejemplo una capacidad de entorno de desarrollo que no exista en producción.

4. Eliminar el fallback "primer tenant activo" en producción.

5. Si no se puede resolver el dominio de forma fiable:

- devolver 404 o respuesta explícita;
- nunca seleccionar un tenant de demostración.

6. Añadir pruebas para:

- dominio correcto;
- dominio inexistente;
- query tenant falso;
- header simulado;
- mezcla de host real + query de otro tenant.

Archivos principales

- `middleware.ts`
- `lib/auth/tenantContext.ts`
- APIs públicas de storefront, productos, media y pedidos.
- tests de resolución de storefront.

Cierre

No existe ningún mecanismo cliente que pueda cambiar el tenant de una petición de producción.

FASE 25 - Prisma fail-closed para aislamiento multi-tenant

Ejecución: AI STUDIO + ORDENADOR DEBIAN
Estado: CRÍTICO

Objetivo

Hacer que PostgreSQL/Prisma rechace una operación tenant-scoped cuando no existe un contexto tenant confiable.

Modificaciones

1. Cambiar la política actual:
 - tenant + bypass válido -> permitir;
 - tenant válido -> aplicar `tenantId`;
 - sin tenant y sin bypass -> rechazar.
2. Diferenciar explícitamente:
 - `SUPER\_ADMIN`;
 - proceso interno de sistema;
 - operación tenant normal.
3. No permitir bypass mediante valores del cliente.
4. Revisar:
 - findMany;
 - findFirst;
 - findUnique;
 - aggregate;
 - groupBy;
 - create;
 - createMany;
 - update;
 - updateMany;
 - upsert;
 - delete;
 - deleteMany.
5. Añadir pruebas específicas para cada tipo de operación.
6. Añadir pruebas de acceso a relaciones y nested writes.

Cierre

Una petición tenant-scoped sin contexto validado falla explícitamente y no puede consultar ni escribir datos globalmente.

FASE 26 - Contrato único de APIs y validación Zod

Ejecución: AI STUDIO
Estado: ALTO

Objetivo

Evitar que cada API tenga su propia interpretación de entrada, tenant, paginación y errores.

Modificaciones

1. Crear esquemas Zod para cada grupo:
 - tenant;
 - products;
 - orders;
 - media;
 - plans;
 - licenses;
 - plugins;
 - themes;
 - domains;
 - checkout;
 - webhooks.
2. Rechazar campos desconocidos en operaciones sensibles cuando corresponda.
3. Definir un formato de error común:
 - `success`;
 - `code`;
 - `message`;

- `details` opcionales;
- `requestId`.

4. Definir límites máximos de:

- `limit`;
- `offset`;
- tamaños de arrays;
- longitud de strings.

5. No usar defaults comerciales peligrosos como:

- tenant demo;
- Madrid;
- precio por defecto;
- producto automático;
- cantidad automática.

Cierre

Las APIs utilizan contratos previsibles y todos los campos comerciales importantes quedan validados en servidor.

FASE 27 - Dinero, precios y precisión comercial

Ejecución: AI STUDIO + ORDENADOR DEBIAN

Estado: CRÍTICO

Objetivo

Eliminar errores de redondeo y manipulación de importes.

Modificaciones

1. Evaluar todos los campos monetarios actuales:

- Plan;
- Payment;
- Invoice;
- Product;
- Order;
- Coupon;
- Subscription.

2. Migrarlos a `Decimal`/`numeric` o a enteros en unidades menores.

3. Definir precisión oficial:

- EUR con 2 decimales en presentación;
- precisión interna definida;
- redondeo en una sola capa.

4. Nunca calcular el total final utilizando un precio enviado por el navegador.

5. En checkout:

- recibir productId y quantity;
- consultar producto en DB;
- consultar precio actual;
- aplicar coupon validado;
- calcular impuesto;
- calcular envío;
- generar total servidor.

6. Guardar en el pedido el precio efectivo utilizado.

Cierre

El importe pagado y facturado puede reconstruirse únicamente a partir de datos persistidos del servidor.

FASE 28 - PayPal Orders API real y captura real

Ejecución: AI STUDIO + pruebas Sandbox

Estado: CRÍTICO

Objetivo

Sustituir completamente las URLs y transacciones ficticias por la API oficial.

Modificaciones

1. Crear servicio `PayPalClient` o equivalente.
2. Obtener OAuth 2.0 access token desde servidor.
3. Seleccionar base URL:
 - Sandbox para pruebas;
 - Live para producción.
4. Crear la orden PayPal desde servidor:
 - importe;
 - moneda;
 - referencia interna;
 - `custom\_id`/metadata interna;
 - return URL;
 - cancel URL.
5. Persistir la relación:
 - Fenix paymentId;
 - PayPal orderId;
 - tenant/request data;
 - estado PENDING.
6. No crear tenant final antes de confirmar el pago.
7. Después de la aprobación:
 - solicitar captura desde servidor.
8. Comprobar respuesta real del proveedor:
 - orderId;
 - captureId;
 - status;
 - importe;
 - currency;
 - referencia interna.
9. Sólo `COMPLETED` permite fulfillment.

Cierre

No existe ninguna ruta de producción que considere suficiente una cadena inventada como `https://www.paypal.com/checkoutnow?token=...`.

FASE 29 - Verificación PayPal y máquina de estados de webhook

Ejecución: AI STUDIO + Sandbox

Estado: CRÍTICO

Objetivo

Hacer que un webhook falso no pueda crear ni renovar recursos.

Modificaciones

1. Mantener el cuerpo original del webhook sin reserializar.
2. Validar:
 - `paypal-transmission-id`;
 - `paypal-transmission-time`;
 - `paypal-cert-url`;
 - `paypal-auth-algo`;
 - `paypal-transmission-sig`;
 - `PAYPAL\_WEBHOOK\_ID`.
3. Implementar:
 - verificación criptográfica completa;
 - o endpoint oficial `verify-webhook-signature`.
4. Cachear certificados si se usa verificación criptográfica directa.

5. Implementar máquina de estados:

- APPROVED;
- PAYMENT.CAPTURE.PENDING;
- PAYMENT.CAPTURE.COMPLETED;
- PAYMENT.CAPTURE.DENIED;
- PAYMENT-APPROVAL.REVERSED;
- REFUNDED;
- eventos de suscripción.

6. Prohibir que `APPROVED` active licencia final por sí solo.

7. Hacer `WebhookEvent.eventId` idempotente.

8. En caso de error de DB al guardar/actualizar el evento:

- no marcarlo como procesado;
- devolver una condición que permita reintento.

Cierre

La autenticidad del mensaje está verificada antes de cambiar cualquier estado comercial.

FASE 30 - Pipeline atómico Pago -> Tenant -> License -> Subscription -> Invoice

Ejecución: AI STUDIO

Estado: CRÍTICO

Objetivo

Convertir la confirmación de pago en una única transición de negocio.

Modificaciones

1. Mantener Payment en PENDING durante checkout.

2. Confirmar el pago con proveedor.

3. Iniciar transacción PostgreSQL.

4. Dentro de la transacción:

- actualizar Payment;
- crear/actualizar Tenant;
- crear/actualizar User owner;
- crear TenantMembership;
- crear License;
- crear LicenseActivation;
- crear/actualizar Subscription;
- crear Invoice.

5. Si falla cualquier paso:

- rollback;
- Payment no debe quedar falseado como completado.

6. Añadir idempotency key de negocio:

- proveedor + order/capture;
- o paymentId interno.

7. Prohibir `ensureTenantExists()` como placeholder comercial antes de pago confirmado.

Cierre

Un pago confirmado produce un estado completo o un rollback completo, nunca un tenant a medias.

FASE 31 - Checkout de tienda, stock y precios protegidos

Ejecución: AI STUDIO

Estado: CRÍTICO

Objetivo

Proteger la venta de productos contra manipulación de precio, stock y tenant.

Modificaciones

1. El pedido público debe derivar tenant exclusivamente del dominio.
2. Ignorar precio enviado por el navegador.
3. Reconsultar:
 - producto;
 - stock;
 - precio;
 - categoría;
 - estado.
4. Validar quantity en servidor.
5. Reservar o bloquear stock en una transacción.
6. Evitar overselling bajo concurrencia.
7. Hacer cupones transaccionales.
8. Guardar snapshot:
 - título;
 - SKU;
 - precio;
 - impuesto;
 - descuento.
9. Rechazar direcciones incompletas en modo real.
10. Eliminar defaults comerciales como "Madrid" cuando el dato es obligatorio.

Cierre

Modificar el JSON del carrito en el navegador no altera el precio final ni permite comprar stock inexistente.

FASE 32 - Storage real, checksum real y seguridad de archivos

Ejecución: AI STUDIO + VPS en integración final

Estado: CRÍTICO

Objetivo

Hacer que la Mediateca almacene bytes reales y pueda recuperarlos tras reinicio.

Modificaciones

1. Definir proveedor de producción:
 - filesystem persistente;
 - S3 compatible;
 - otro storage duradero.
2. ``upload()`` debe escribir realmente los bytes.
3. El checksum debe calcularse sobre los bytes del archivo.
4. Eliminar ``delete()`` { return true; } como implementación final.
5. Un upload sólo puede devolver éxito después de:
 - storage OK;
 - DB OK.
6. Si DB falla después del upload:
 - borrar el objeto;
 - o registrarlo para reconciliación.
7. Validar MIME mediante contenido/magic bytes cuando sea posible.
8. Evitar SSRF en URLs remotas de media.
9. Crear URLs seguras:
 - públicas sólo cuando corresponda;
 - signed URLs cuando el recurso sea privado.
10. Unificar el límite de tamaño.

Cierre

Subir un archivo, reiniciar el servicio y volver a abrir la Mediateca devuelve el mismo archivo real.

FASE 33 - Plugins y temas como paquetes seguros

Ejecución: AI STUDIO

Estado: ALTO/CRÍTICO

Objetivo

Transformar el sistema de plugins/temas de "manifest + registro" a un sistema real de paquetes gobernados.

Modificaciones

1. Separar:

- catálogo global;
- instalación por tenant.

2. Definir manifest versionado.

3. Validar:

- nombre;
- versión;
- aplicación compatible;
- CMS mínimo;
- dependencias;
- permisos;
- hooks;
- archivos permitidos.

4. Para ZIP:

- impedir path traversal;
- bloquear archivos ejecutables no permitidos;
- limitar número de archivos;
- limitar tamaño descomprimido;
- validar tipos MIME;
- impedir symlinks peligrosos.

5. No considerar suficiente el escaneo de cadenas `eval()`.

6. Definir un modelo de ejecución:

- plugins declarativos y hooks seguros;
- o sandbox/proceso aislado si se permite código ejecutable.

7. Firmar versiones publicadas cuando el producto evolucione a un marketplace de terceros.

Cierre

Un archivo ZIP no se ejecuta ni se instala sólo porque el manifest tenga un formato correcto.

FASE 34 - Autenticación, CSRF, cookies y rate limiting distribuido

Ejecución: AI STUDIO

Estado: ALTO

Objetivo

Endurecer la superficie de autenticación y las operaciones que modifican datos.

Modificaciones

1. Definir protección CSRF/origin para peticiones basadas en cookie.

2. Mantener cookies:

- HttpOnly;
- Secure en producción;
- SameSite adecuado.

3. Evaluar nombre de cookie con prefijo `Host-` si la arquitectura lo permite.

4. Rotar sesión cuando cambian privilegios sensibles.

5. Revocar sesiones tras:

- cambio importante de contraseña;
- suspensión;

- cambio de privilegio.

6. No confiar ciegamente en `x-forwarded-for`.

7. Definir proxy de confianza.

8. Sustituir el `Map` de rate limiting por un mecanismo compartido:

- Redis;
- PostgreSQL apropiadamente diseñado;
- infraestructura equivalente.

9. Añadir límites distintos para:

- login;
- cambio de contraseña;
- checkout;
- webhooks;
- media;
- API general.

Cierre

La protección de abuso no depende de una sola instancia Node y las operaciones de escritura basadas en cookie están protegidas contra solicitudes de origen no autorizado.

FASE 35 - Eliminar todos los fallback de demo en producción

Ejecución: AI STUDIO

Estado: CRÍTICO

Objetivo

Que un error no se convierta silenciosamente en una tienda ficticia.

Modificaciones

1. Buscar todos los `INITIAL\_\*`.

2. Clasificarlos:

- test;
- development;
- documentation;
- production.

3. Eliminar de producción los fallback de:

- plans;
- tenants;
- products;
- users;
- media;
- invoices;
- domains;
- plugins;
- themes.

4. Corregir patrones:

- `catch -> return INITIAL\_\*`;
- `DB empty -> demo`;
- `DB error -> success:true`.

5. Para DB vacía:

- devolver lista vacía cuando sea válido;
- no inventar contenido.

6. Para error:

- devolver error controlado.

Cierre

Una caída o inconsistencia de PostgreSQL nunca presenta datos demo como datos reales.

FASE 36 - Super Admin completamente persistente

Ejecución: AI STUDIO
Estado: CRÍTICO

Objetivo

Convertir el panel Super Admin en una consola operativa real.

Modificaciones

1. Todos los listados de Super Admin deben venir de PostgreSQL.
2. Diferenciar:
 - cero registros;
 - error de consulta.
3. Persistir `PlatformSettings`.
4. Completar endpoints de entitlements:
 - GET;
 - POST;
 - PATCH/PUT;
 - DELETE si procede.
5. Añadir validaciones de dependencia antes de borrar:
 - plan usado por tenant;
 - plan usado por licencia;
 - entitlement asociado.
6. Todas las mutaciones deben registrar AuditLog.
7. No incluir claves de licencia completas ni PII innecesaria en respuestas de listados.

Cierre

Cerrar sesión y volver a abrir Super Admin conserva exactamente el estado de la plataforma.

FASE 37 - Integridad de base de datos, constraints e índices

Ejecución: AI STUDIO + ORDENADOR DEBIAN
Estado: ALTO

Objetivo

Hacer que PostgreSQL proteja datos críticos, no sólo el código.

Modificaciones

1. Revisar constraints de:
 - Payment;
 - Invoice;
 - License;
 - Subscription;
 - Order;
 - Product;
 - Domain.
2. Revisar unique keys por tenant.
3. Evaluar:
 - SKU;
 - barcode;
 - orderNumber;
 - idempotencyKey;
 - domain hostname;
 - plugin key;
 - theme slug.
4. Revisar índices según consultas reales.
5. Añadir índices compuestos donde sean necesarios por tenant.
6. Introducir soft delete/archive en registros donde la eliminación física pueda destruir trazabilidad comercial.
7. Crear migraciones pequeñas y reversibles.

Cierre

La base de datos impide estados imposibles y las restricciones importantes están expresadas en PostgreSQL.

FASE 38 - Pooling, timeouts y conexiones PostgreSQL

Ejecución: ORDENADOR DEBIAN + VPS

Estado: ALTO

Objetivo

Preparar PostgreSQL para tráfico real y procesos concurrentes.

Modificaciones

1. Revisar número de conexiones de Prisma.
2. Definir URL de aplicación con pool adecuado.
3. Separar conexión de aplicación y conexión directa para migraciones si la infraestructura lo necesita.
4. Configurar timeouts.
5. Revisar consultas lentas.
6. Medir:
 - p50;
 - p95;
 - p99;
 - número de conexiones;
 - errores por timeout.
7. No aumentar el pool de forma arbitraria.

Cierre

Existe una configuración documentada y comprobada de conexiones para aplicación y migraciones.

FASE 39 - CI/CD profesional en GitHub

Ejecución: GITHUB

Estado: CRÍTICO

Objetivo

Que ningún cambio importante llegue a `main` sin ser probado.

Modificaciones

Crear un workflow principal, por ejemplo:

`.github/workflows/ci.yml`

que ejecute:

1. Checkout.
2. Node 22 según `.nvmrc`.
3. `npm ci`.
4. TypeScript.
5. ESLint.
6. Prisma validate.
7. Tests.
8. Build Next.js.
9. Comprobaciones de seguridad.
10. Publicación del resultado.

Añadir además, de forma progresiva:

- CodeQL;
- Dependabot;
- revisión de dependencias;
- comprobación de secretos;
- cobertura si el proyecto se estabiliza.

Reglas de GitHub

Proteger `main` con reglas adecuadas:

- Pull Request;
- status checks obligatorios;
- revisión;
- resolución de conversaciones;
- bloqueo de force push;
- restricciones de bypass.

GitHub permite configurar estas exigencias mediante protección de ramas y rulesets.

Cierre

Un cambio que rompe build/tests no puede convertirse automáticamente en release de producción.

FASE 40 - Unificar Node/npm como herramienta reproducible

Ejecución: ORDENADOR DEBIAN + GITHUB

Estado: ALTO

Objetivo

Que Debian, GitHub Actions y el entorno de desarrollo utilicen el mismo procedimiento.

Modificaciones

1. Decidir `npm` como herramienta estándar de CI y desarrollo compatible con el equipo actual.
2. Añadir `packageManager` en `package.json`.
3. Mantener un único lockfile oficial.
4. Adaptar `scripts/run-all-tests.ts` para no depender exclusivamente de `bunx`.
5. Añadir scripts explícitos:
 - `typecheck`;
 - `lint`;
 - `test`;
 - `build`;
 - `db:validate`;
 - `db:migrate`;
 - `db:seed` cuando corresponda.
6. Verificar que los comandos funcionan en:
 - Debian;
 - GitHub Actions;
 - VPS.

Cierre

El mismo commit se puede instalar y probar con el mismo procedimiento en todos los entornos.

FASE 41 - Observabilidad y trazabilidad completa

Ejecución: AI STUDIO + VPS

Estado: ALTO

Objetivo

Poder responder qué ocurrió en un pago, login, pedido o error.

Modificaciones

1. Crear `requestId` por petición.
2. Incluirlo en logs.
3. AuditLog para eventos críticos:
 - login;
 - logout;
 - cambio de rol;
 - cambio de plan;
 - licencia;

- pago;
- refund;
- webhook;
- dominio;
- plugin;
- tema.

4. Nunca registrar secretos.

5. Añadir métricas:

- error rate;
- latency;
- DB latency;
- webhooks;
- payments;
- checkout;
- uploads.

6. Añadir health:

- liveness;
- readiness;
- dependencias.

Cierre

Una incidencia puede seguirse desde request hasta AuditLog y, cuando exista, hasta Payment/WebhookEvent.

FASE 42 - Privacidad, datos personales y facturación

Ejecución: AI STUDIO + revisión del negocio

Estado: ALTO

Objetivo

Preparar el SaaS para gestionar datos reales de clientes.

Modificaciones

1. Inventariar PII:

- email;
- nombre;
- teléfono;
- dirección;
- IP;
- user agent;
- datos de facturación.

2. Definir retención.

3. Evitar guardar datos que no sean necesarios.

4. Diseñar exportación de datos del tenant cuando corresponda.

5. Diseñar eliminación/anonimización con conservación de trazabilidad legal necesaria.

6. Revisar invoices:

- nombre fiscal;
- dirección;
- país;
- identificador fiscal;
- numeración;
- moneda;
- impuestos.

7. Separar configuración técnica de decisión fiscal/legal.

Cierre

Existe un mapa de datos personales y una política técnica de retención y eliminación.

FASE 43 - Rendimiento, cache y escalabilidad multi-tenant

Ejecución: AI STUDIO + pruebas de carga
Estado: ALTO

Objetivo

Que el crecimiento de tenants no degrade el storefront ni el backoffice.

Modificaciones

1. Auditar N+1.
2. Paginar todas las colecciones grandes.
3. Limitar includes Prisma demasiado grandes.
4. Definir cache por:
 - hostname;
 - tenantId;
 - recurso;
 - versión.
5. Invalidar cache después de:
 - cambio de tema;
 - branding;
 - producto;
 - dominio;
 - publicación de página.
6. Optimizar imágenes.
7. Medir consultas PostgreSQL lentas.
8. Hacer pruebas concurrentes con varios tenants.

Cierre

El cache nunca mezcla tenants y una invalidación deja visibles rápidamente los datos nuevos.

FASE 44 - Aceptación final, recuperación y go-live

Ejecución: ORDENADOR DEBIAN + GITHUB + VPS
Estado: FINAL

Objetivo

Realizar una aceptación real antes de considerar FenixCMS listo para clientes.

Paso 1 - Staging

Crear un entorno de staging con:

- PostgreSQL independiente;
- credenciales Sandbox;
- dominio de staging;
- storage de staging;
- secrets separados.

Paso 2 - Pruebas SaaS

Comprobar:

- alta de tenant;
- login;
- cambio de tenant;
- plan;
- entitlement;
- licencia;
- suscripción;
- factura.

Paso 3 - Pruebas PayPal

Comprobar:

- create order;
- approve;

- capture;
- webhook;
- duplicado;
- pending;
- denied;
- refund.

Paso 4 - Pruebas multi-tenant

Comprobar como mínimo:

- Tenant A no puede leer Tenant B;
- Tenant B no puede leer Tenant A;
- ninguna URL/query/header permite cambiar tenant en producción;
- Super Admin sí puede operar mediante rutas autorizadas.

Paso 5 - Pruebas de recuperación

Simular:

- PostgreSQL caído;
- storage caído;
- webhook repetido;
- reinicio Node;
- error en una transacción;
- restauración de backup.

Paso 6 - Backup

Comprobar no sólo que existe backup, sino que se puede restaurar en un entorno independiente.

Paso 7 - Go-live gate

No desplegar a producción hasta que:

- CI pase;
- migraciones pasen;
- tests críticos pasen;
- PayPal Sandbox complete el flujo;
- staging pase;
- backup restore pase;
- auditoría de seguridad pase;
- no existan fallback demo en producción.

Cierre

El sistema queda preparado para producción real con una evidencia reproducible y documentada de las pruebas.

5. Matriz de archivos principales a revisar

Seguridad y contexto

- `middleware.ts`
- `lib/auth/tenantContext.ts`
- `lib/auth/session.ts`
- `lib/auth/admin-guard.ts`
- `lib/auth/rbac.ts`
- `lib/security/security.service.ts`
- `lib/prisma.ts`

Pago

- `lib/services/payment.service.ts`
- `lib/services/saas-checkout.service.ts`
- `lib/services/webhook.service.ts`
- `app/api/billing/checkout/route.ts`
- `app/api/billing/verify/route.ts`
- `app/api/webhooks/paypal/route.ts`
- `app/api/webhooks/stripe/route.ts`
- `app/api/tenants/provision/route.ts`
- `components/saas/SaaS Landing.tsx`

Persistencia CMS

- `lib/storeContext.tsx`
- `lib/services/super-admin.service.ts`
- `lib/services/product.service.ts`
- `lib/services/order.service.ts`
- `lib/services/storefront.service.ts`
- `lib/storage/storage.service.ts`
- `app/api/media/route.ts`
- `app/api/products/route.ts`
- `app/api/orders/route.ts`

Modelo de datos

- `prisma/schema.prisma`
- `prisma/seed.ts`
- `prisma/migrations/\*`

Calidad y CI

- `package.json`
- `package-lock.json`
- `bun.lock`
- `scripts/run-all-tests.ts`
- `tests/\*`
- `.github/workflows/\*`

6. Regla de trabajo con AI Studio

Para cada fase que se entregue a AI Studio:

1. AI Studio debe analizar primero el estado real actual.
2. Debe modificar sólo lo perteneciente a la fase.
3. Debe conservar funcionalidades válidas existentes.
4. No debe introducir datos demo como fallback de producción.
5. Debe crear o actualizar tests de la propia fase.
6. Debe ejecutar build y tests.
7. Debe informar:
 - archivos modificados;
 - migraciones creadas;
 - tests ejecutados;
 - resultados;
 - riesgos restantes.
8. No debe declarar una fase terminada sólo porque la interfaz visual funcione.
9. La persistencia debe verificarse desde PostgreSQL cuando la fase toque datos.
10. Después del informe de AI Studio se revisará el diff antes de avanzar.

7. Definición de terminado para cualquier fase

Una fase sólo puede marcarse como terminada cuando se cumplen simultáneamente:

Código

- Implementación presente.
- Tipos correctos.
- Sin errores de build.

Persistencia

- PostgreSQL guarda el dato cuando corresponde.
- Reiniciar la aplicación no elimina el resultado.

Seguridad

- Tenant correcto.
- Rol correcto.
- Entitlement correcto.
- Sin bypass cliente.

Pruebas

- Caso normal.
- Caso inválido.

- Caso de seguridad.
- Caso de fallo/rollback cuando corresponda.

Operación

- Logs suficientes.
- Error controlado.
- Sin secretos expuestos.

8. Referencias técnicas externas utilizadas

PayPal - Orders API y flujo server-side:

<https://developer.paypal.com/api/rest/integration/orders-api>

PayPal - Webhooks:

<https://developer.paypal.com/api/rest/webhooks>

PayPal - Integración y verificación de webhooks:

<https://developer.paypal.com/api/rest/webhooks/rest/>

PayPal - Webhooks de Checkout:

<https://developer.paypal.com/api/rest/webhooks/event-names/>

Prisma - Schema y Decimal:

<https://www.prisma.io/docs/orm/v6/reference/prisma-schema-reference>

GitHub - Protección de ramas:

<https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches>

GitHub - Rulesets:

<https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-rulesets/available-rules-for-rulesets>

9. Resultado esperado al terminar las FASES 24-44

Al finalizar estas fases, FenixCMS deberá haber pasado de una arquitectura con partes ya sólidas y otras aún híbridas a una arquitectura donde:

- PostgreSQL sea la autoridad real.
- El tenant se determine por contexto confiable.
- No exista fail-open de aislamiento.
- El dinero use precisión adecuada.
- PayPal utilice su API real.
- La captura real sea la condición de fulfillment.
- Los webhooks estén autenticados e idempotentes.
- Pago, licencia, suscripción, factura y tenant sean coherentes.
- El checkout de tienda no acepte precios manipulados.
- Media y storage sean reales.
- Plugins y temas tengan un modelo de instalación seguro.
- Super Admin sea persistente.
- CI bloquee cambios defectuosos.
- El entorno de Debian, GitHub y VPS sea reproducible.
- Existan logs, métricas, backup y recuperación.
- Staging sea una barrera antes de producción.

IMPORTANTE: este documento no autoriza todavía el despliegue final. El paso correcto continúa siendo ejecutar las fases una a una y validar cada una antes de pasar a la siguiente.

Control de generación GitHub Actions: documento preparado para generación automática del PDF en `main`.